GOVERNMENT ENGINEERING COLLEGE IDUKKI

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING

CSL 332 NETWORKING LAB

Record Submitted by

Aashin N Aneeshkumar (IDK23CS002)

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY
THIRUVANANTHAPURAM

March 2026

GOVERNMENT ENGINEERING COLLEGE
IDUKKI

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING

CSL 332 NETWORKING LAB RECORD

Certified that this is a bonafide record of the work done by Sri/Smt Aashin N Aneeshkumar (IDK23CS002) of the Sixth Semester Computer Science Engineering class in the CSL 332 Networking Lab during the year 2025–26.

Staff in Charge

Internal Examiner **External Examiner**

INDEX

EXP NO : 1 **Date : 03/12/2025**

Familiarization of Linux networking commands

Aim

To familiarize with basic networking concepts and commonly used networking commands.

Q1 (a). What is nslookup? Use nslookup to find the ip address of the following domain names :

- www.gecidukki.ac.in
- www.iitm.ac.in
- www.iitb.ac.in
- www.google.com

Answer: The nslookup is a command-line administration tool used to query the Domain Name System (DNS) to obtain domain name or IP address mapping, and other specific DNS records.

1. **www.gecidukki.ac.in**

   Command:

   ```
   nslookup www.gecidukki.ac.in
   ```

   Output:

   ```
   canonical name = gecidukki.ac.in
   Address: 198.7.125.132
   ```

   Result: The IP address of www.gecidukki.ac.in is 198.7.125.132.

2. **www.iitm.ac.in**

   Command:

`nslookup www.iitm.ac.in`

Output:

```
Address: 103.158.42.45
```

Result: The IP address of www.iitm.ac.in is 103.158.42.45.

3. **www.iitb.ac.in**

   Command:

   `nslookup www.iitb.ac.in`

   Output:

   ```
   Address: 103.21.124.133
   ```

   Result: The IP address of www.iitb.ac.in is 103.21.124.133.

4. **www.google.com**

   Command:

   `nslookup www.google.com`

   Output:

   ```
   Address: 142.251.154.119
   ```

   Result: The IP address of www.google.com is 142.251.154.119.

Q1 (b). Visit http://whois.icann.org and find the location of the servers hosting the above domain names. Prepare a table as follows:

Domain name	IP Address	Server location
www.gecidukki.ac.in	198.7.125.132	IN
www.iitm.ac.in	103.158.42.45	IN
www.iitb.ac.in	103.21.124.133	IN
www.google.com	142.251.154.119	US

Q2. What is ifconfig? Use ifconfig to find the live interface on your Computer. Fill in the following table with its information.

Answer: The 'ifconfig' is a command-line utility used to view, configure, and troubleshoot network interfaces on Unix-like operating systems, including Linux and macOS.

- Network Interface Information :

Type	IPv4 Address	Netmask	MAC Address	IPv6 Address
Wired	172.28.107.34	255.255.240.0	00:15:5d:7c:e1:28	fe80::215:5dff:fe7c:e128
Loop Back	127.0.0.1	255.0.0.0	N/A	:: 1
Wi-Fi	N/A	N/A	0C:54:15:32:91:50	N/A

Q3. What is ping? Use ping to fill in the following table for the domain names :

- www.gecidukki.ac.in
- www.iitm.ac.in
- www.iitb.ac.in
- www.google.com

Answer: 'ping' is a standard command-line utility used to test the reachability of a host on an IP network and to measure the round-trip time (RTT) for messages sent from the originating host to a destination computer.

Domain Name	IP Address	Average RTT(ms)	TTL(ms)	Packet loss%
www.iitm.ac.in	103.158.42.45	–	–	100%
www.iitb.ac.in	103.21.124.133	–	–	100%
www.gecidukki.ac.in	198.7.125.132	270.699	109	0%
www.google.com	142.251.154.119	49.201	116	0.84%

Q4. What is traceroute? Use traceroute to find the route to the following servers. Fill in the following table.

- www.gecidukki.ac.in
- www.iitm.ac.in
- www.iitb.ac.in
- www.google.com

Answer: 'traceroute' is a diagnostic utility used to track the real-time path that data packets take from a source to a destination.

Domain Name	Number of hops	Max RTT(ms)
gecidukki.ac.in	23	29.15
iitm.ac.in	10	59.5
iitb.ac.in	18	22.83
google.com	12	39

Result

The networking commands were studied and understood successfully.

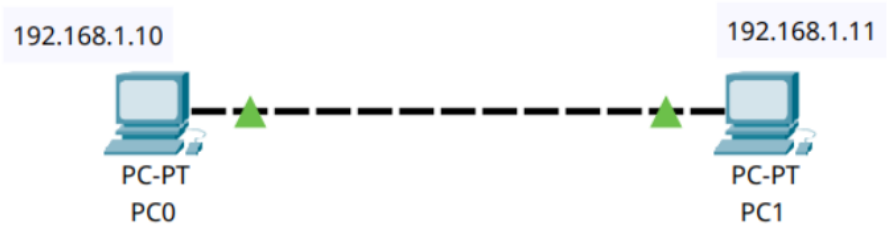
EXP NO : 2 **Date : 17/12/2025**

Simulation of Networks and Network Services using CISCO Packet Tracer

Aim

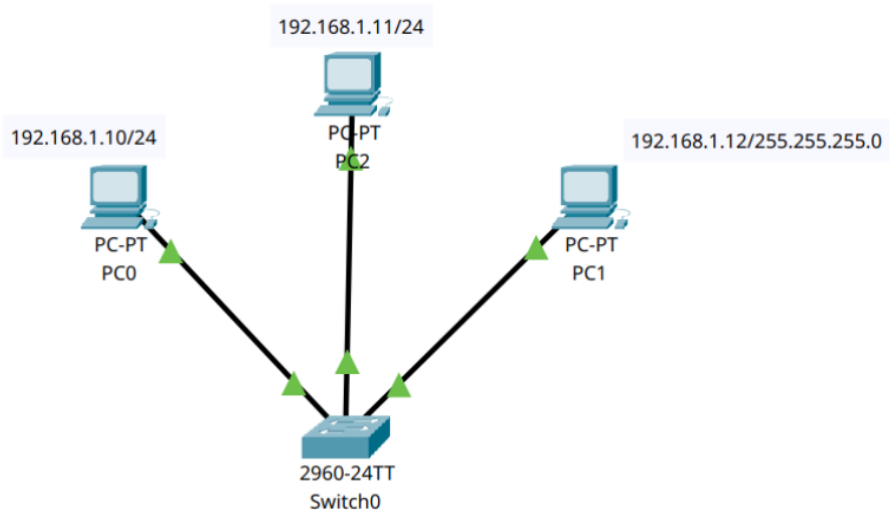
A. To simulate wired networks using different network devices and configure end devices to ensure connectivity.

1. **Direct connection between 2 end devices**



2. **Interconnect 3 computers using Star topology**

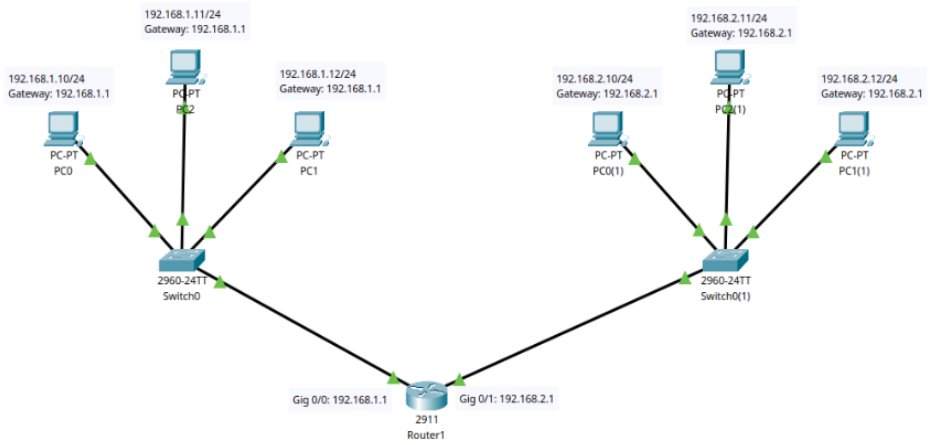
   Network device used: Switch



3. **Communication between two networks using routers**

   Network 1 192.168.1.0

   Network 2 192.168.2.0



B. To simulate Network Services on Cisco Packet Tracer.

1. Simulating DHCP Server

- Create a network using 3 computers and one server using a switch.
- Assign IP address 192.168.1.1 to the server.
- Choose DHCP from the services tab on the server.
- Turn on the sevice.
- Set the start IP address to 192.168.1.10 in the server pool.
- Set the maximum number of users to 10.
- Click the save button.
- Set the IP configuration of each machine to DHCP.
- Confirm that the new IP adress is assigned automatically from the DHCP server.

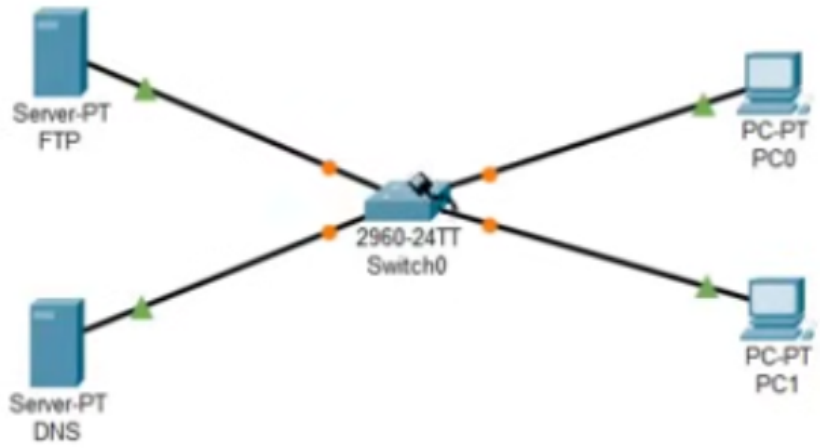
2. Simulating Email Server

- We will set the Email server on top of the DHCP server.
- Ensure that SMTP and POP3 service are on on the email server.
- Set Domain name to gecmail.com and click set.
- In User set up, create users and add them using the + symbol
- name: final1, password: 123
- name: final2, password: 123
- In the first client, choose Email from the Desktop.

- In configure Email, add a name and choose final1@gecmail.com
- In server information, use 192.168.1.1 for both incoming and outgoing mail servers.
- Logon Information final1 and 123 and save.
- Repeat the client steps with final2@gecmail.com for the second client.
- Compose email and sent to final1@gecmail.com
- Check the first client and ensure that the message is received.

3. Simulating FTP Server

- Configure network as shown below.



- DNS 192.168.1.1. Also set the same IP as DNS server.
- FTP 192.168.1.100. Set 192.168.1.1 as DNS server.
- PC0 192.168.1.2. Set 192.168.1.1 as DNS server.
- PC1 192.168.1.3. Set 192.168.1.1 as DNS server.
- In the FTP server,
  - turn on FTP
  - add a new user, user1 and set password to 123.
  - Give Read and Write permission to the user.
- In PC0, create a new text file, newFile, from the text editor.
  - Type something to newFile and save it.
  - Login to the FTP server from command prompt
    - ftp 192.18.1.100
  - Use put command to copy the file to the FTP server.

    - put file1.txt.
  - Exit from FTP service.
- Login to the FTP server from PC1 and copy the file to PC1 using get command.
  - ftp 192.18.1.100
  - get file1.txt.

Result

- Verified connectivity using ping between devices.
- Successfully implemented DHCP, SMTP and FTP services using Cisco Packet Tracer.

EXP NO : 3 **Date :17/12/2025**

Familiarization of Linux System Calls

Aim

Familiarization of system calls used for network programming in Linux.

Introduction

In Linux, network communication between processes is achieved using sockets. A socket acts as an endpoint for communication between two machines over a network. Two major transport layer protocols used are:

- Transmission Control Protocol (TCP) : Connection-oriented, reliable communication.
- User Datagram Protocol (UDP) : Connectionless, faster but unreliable communication

Both use the Berkeley Socket API, available in Linux through system calls.

TCP Communication

1. **TCP provides**
   - Reliable data transfer
   - Error detection and correction
   - Flow control
   - Ordered delivery

   TCP requires connection establishment before communication.

2. **Important TCP System Calls**
   - socket() :
     Creates a socket.
     int socket(int domain, int type, int protocol);

  - AF_INET → IPv4
  - SOCK_STREAM → TCP

- bind() :
  Associates socket with IP address and port number.
  int bind(int sockfd, struct sockaddr *addr, socklen_t addrlen);
- listen() :
  Marks socket as passive (waiting for connections).
  int listen(int sockfd, int backlog);
- accept() :
  Accepts incoming connection request.
  int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
- connect() :
  Used by client to establish connection.
- send()/write() :
  Sends data over TCP.
- recv()/read() :
  Receives data over TCP.
- close() :
  Closes socket connection.

3. Flow of TCP Communication

Server Side

- socket() → bind() → listen() → accept() → recv()/send() → close()

Client Side

- socket() → connect() → send()/recv() → close()

UDP Communication

1. UDP provides

- Fast transmission
- No connection establishment
- No guarantee of delivery
- No ordering

UDP is suitable for real-time applications.

2. Important UDP System Calls

- socket() :
  socket(AF_INET, SOCK_DGRAM, 0)
  SOCK_DGRAM → UDP
- bind() :
  Used by server to attach port number.
- sendto() :
  Sends data to specific address.
- recvfrom() :
  Receives data and sender address.
- close() :
  Closes the socket.

3. Flow of UDP Communication

Server Side

- socket() → bind() recvfrom() → sendto() → close()

Client Side

- socket() → sendto() → recvfrom() → close()

Result

The UDP and TCP system calls are familarized.

EXP NO : 3A **Date : 31/12/2025**

TCP Echo Server

Aim

Implementation of TCP echo server.

Algorithm

Server :

1. Start
2. Create a socket using `socket()` with `SOCK_STREAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5000`, `INADDR_ANY`).
5. Bind the socket to the address using `bind()`.
6. Check for binding error.
7. Listen for incoming connections using `listen()` with backlog 4.
8. Check for listening error.
9. Accept a client connection using `accept()`.
10. Check for accept error.
11. Receive message from client into buffer using `recv()`.
12. Check for receiving error.
13. Print the received message.
14. Send response `"Hello from the server!"` using `send()`.
15. Check for sending error.
16. Close client socket using `close(clientfd)`.
17. Close server socket using `close(sockfd)`.
18. Stop

Client :

1. Start
2. Create a socket using `socket()` with `SOCK_STREAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5000`, IP `127.0.0.1`).
5. Connect to the server using `connect()`.
6. Check for connection error.
7. Print `"Server connection success"`.
8. Set message to `"Hello from client"`.
9. Send message to server using `send()`.
10. Check for sending error.
11. Print `"Message sent"`.
12. Receive response from server into buffer using `recv()`.
13. Check for receiving error.
14. Print the received message from server.
15. Close socket using `close(sockfd)`.
16. Stop

Program Code

Server :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h> // htons, htonl, ntohs, ntohl

int main() {
    struct sockaddr_in server_addr, client_addr;
```

```
int sockfd, clientfd;
if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    perror("Error creating socket!\n");
    exit(1);
}
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(5000); // host to network short
   used to convert the 16bit unsigned integer to network byte
    order
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
socklen_t server_addr_len = sizeof(server_addr);
if((bind(sockfd, (struct sockaddr *)&server_addr,
   server_addr_len)) < 0) {
    perror("Error binding!\n");
    exit(1);
}
int backlogs = 4; // maximum no of connection the kernel
   should queue for this socket
if((listen(sockfd, backlogs)) < 0) {
    perror("Error listening!\n");
    exit(1);
}
// Client connection accept
socklen_t client_addr_len = sizeof(client_addr);
clientfd = accept(sockfd, (struct sockaddr *)&client_addr, &
   client_addr_len);
if(clientfd < 0) {
    perror("Error while accept!\n");
    exit(1);
}
printf("Client connection success\n");
// Receive data from client
char buffer[1024];
if((recv(clientfd, buffer, sizeof(buffer) - 1, 0)) < 0) {
    perror("Error while receiving data!\n");
    exit(1);
}
printf("Message from client: %s\n", buffer);
// Send a response to the client
char *response = "Hello from the server!";
if (send(clientfd, response, strlen(response), 0) < 0) {
```

```
        perror("Error sending response to client");
    }
    printf("Sent response to client\n");
    // Close the connections
    close(clientfd);
    close(sockfd);
    return 0;
}
```

Client :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h> // inet_addr

int main() {
    struct sockaddr_in server_addr;
    int sockfd;
    if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Error creating socket!\n");
        exit(1);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5000); // host to network short
       used to convert the 16bit unsigned integer to network byte
        order
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    socklen_t server_addr_len = sizeof(server_addr);
    if((connect(sockfd, (struct sockaddr *)&server_addr,
       server_addr_len)) < 0) {
        perror("Error connecting to server!\n");
        exit(1);
    }
    printf("Server connection success\n");
    // Send a message to the server
    char *msg = "Hello from client";
    if((send(sockfd, msg, strlen(msg), 0)) < 0) {
        perror("Error while sending message!\n");
```

```
        exit(1);
    }
    printf("Message sent\n");
    // Receive data from server
    char buffer[1024];
    if((recv(sockfd, buffer, sizeof(buffer) - 1, 0)) < 0) {
        perror("Error while receiving data!\n");
        exit(1);
    }
    printf("Message from server: %s\n", buffer);
    return 0;
}
```

Output

Server :

```
Client connection success
Message from client: Hello from client
Sent response to client
```

Client :

```
Server connection success
Message sent
Message from server: Hello from the server!
```

Result

The Program was executed successfully and output is verified.

EXP NO : 3B **Date : 31/12/2025**

UDP Echo Server

Aim

Implementation of UDP echo server

Algorithm

Server

1. Start
2. Create a socket using `socket()` with `SOCK_DGRAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5001`, `INADDR_ANY`).
5. Bind the socket using `bind()`.
6. Check for binding error.
7. Receive message from client using `recvfrom()`.
8. Check for receiving error.
9. Print received client data.
10. Send response to client using `sendto()`.
11. Check for sending error.
12. Print server data sent confirmation.
13. Close socket.
14. Stop

Client

1. Start
2. Create a socket using `socket()` with `SOCK_DGRAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5001`, IP `127.0.0.1`).
5. Send message to server using `sendto()`.
6. Check for sending error.
7. Print client data sent confirmation.
8. Receive response from server using `recvfrom()`.
9. Check for receiving error.
10. Print received server data.
11. Close socket.
12. Stop

Program Code

Server :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main() {
    struct sockaddr_in server_addr;
    int sockfd;

    if((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket connection failed!\n");
        exit(1);
    }
```

```
    memset(&server_addr, '\0', sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5001);
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);

    socklen_t server_addr_len = sizeof(server_addr);
    if((bind(sockfd, (struct sockaddr *)&server_addr,
       server_addr_len)) < 0) {
        perror("Error binding connection!\n");
        exit(1);
    }

    char buffer[1024] = {0};
    if((recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct
       sockaddr *)&server_addr, &server_addr_len)) < 0) {
        perror("Error receiving content/data from client!\n");
        exit(1);
    }
    printf("[+] Client data received: %s\n", buffer);

    char *msg = "Message from server";
    if((sendto(sockfd, msg, strlen(msg), 0, (struct sockaddr *)&
       server_addr, server_addr_len)) < 0) {
        perror("Error sending content/data to client!\n");
        exit(1);
    }
    printf("[+] Server data sent\n");

    return 0;
}
```

Client :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
```

```
int main() {
    struct sockaddr_in server_addr;
    int sockfd;

    if((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket connection failed!\n");
        exit(1);
    }

    memset(&server_addr, '\0', sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5001);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    socklen_t server_addr_len = sizeof(server_addr);
    char *msg = "Message from client";
    if((sendto(sockfd, msg, strlen(msg), 0, (struct sockaddr *)&
       server_addr, server_addr_len)) < 0) {
        perror("Error sending content/data to server!\n");
        exit(1);
    }
    printf("[+] Client data sent\n");

    char buffer[1024] = {0};
    if((recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct
       sockaddr *)&server_addr, &server_addr_len)) < 0) {
        perror("Error receiving content/data from server!\n");
        exit(1);
    }
    printf("[+] Server data received: %s\n", buffer);

    return 0;
}
```

Output

Server :

```
[+] Client data received: Message from client
[+] Server data sent
```

Client :

```
[+] Client data sent
[+] Server data received: Message from server
```

Result

The Program was executed successfully and output is verified.

EXP NO : 4 **Date : 31/12/2025**

TCP Client - Server Implementation

Aim

Write a TCP client-server program with the following functionality:
A. The client reads a string as input from the user and sends it to the server.
B. The server reverses the string and sends it back to the client.

Algorithm

Server :

1. Start
2. Create a socket using `socket()` with `SOCK_STREAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5000`, `INADDR_ANY`).
5. Bind the socket to the address using `bind()`.
6. Check for binding error.
7. Listen for incoming connections using `listen()` with backlog 4.
8. Check for listening error.
9. Accept a client connection using `accept()`.
10. Check for accept error.
11. Receive message from client into buffer using `recv()`.
12. Check for receiving error.
13. Print the received string.
14. Reverse the received string using `revString()`.
15. Send the reversed string back to the client using `send()`.
16. Check for sending error.

17. Close client socket using `close(clientfd)`.
18. Close server socket using `close(sockfd)`.
19. Stop

Client :

1. Start
2. Create a socket using `socket()` with `SOCK_STREAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5000`, IP `127.0.0.1`).
5. Connect to the server using `connect()`.
6. Check for connection error.
7. Print `"Server connection successfull"`.
8. Read a message from the user using `fgets()`.
9. Send the message to the server using `send()`.
10. Check for sending error.
11. Clear the message buffer using `memset()`.
12. Receive the reversed string from the server using `recv()`.
13. Check for receiving error.
14. Print the reversed string.
15. Close socket using `close(sockfd)`.
16. Stop

Program Code

Server :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/socket.h>

void revString(char *str) {
    int i = 0, j = 0;
    char temp;

    // Find length of string
    j = strlen(str);
    j--; // last character index

    // Swap characters from both ends
    while (i < j) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
        i++;
        j--;
    }
}

int main() {
    struct sockaddr_in server_addr, client_addr;
    int sockfd, clientfd;

    if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket connection failed!\n");
        exit(1);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5000);
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);

    socklen_t server_addr_len = sizeof(server_addr);
    if((bind(sockfd, (struct sockaddr *)&server_addr,
```

```
       server_addr_len)) < 0) {
        perror("Bind connection failed!\n");
        exit(1);
    }

    if ((listen(sockfd, 4)) < 0) {
        perror("Listen failed!\n");
        exit(1);
    }

    socklen_t client_addr_len = sizeof(client_addr);
    if((clientfd = accept(sockfd, (struct sockaddr *)&client_addr
       , &client_addr_len)) < 0) {
        perror("Accept failed!\n");
        exit(1);
    }

    printf("Client connection successful\n");
    char buffer[1024] = {'\0'};
    if((recv(clientfd, buffer, sizeof(buffer), 0)) < 0) {
        perror("Error receiving content from client");
        exit(1);
    }

    printf("Received string: %s\n", buffer);
    revString(buffer);

    if((send(clientfd, buffer, strlen(buffer), 0)) < 0) {
        perror("Error while sending message\n");
        exit(1);
    }

    close(sockfd);
    close(clientfd);
    return 0;
}
```

Client :

```
#include <stdio.h>
#include <string.h>
```

```
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/socket.h>

int main() {
    struct sockaddr_in server_addr;
    int sockfd;

    if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket connection failed!\n");
        exit(1);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5000);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    socklen_t server_addr_len = sizeof(server_addr);
    if((connect(sockfd, (struct sockaddr *) &server_addr,
       server_addr_len)) < 0) {
        printf("Error connecting server");
        exit(1);
    }

    printf("Server connection successful\n");
    char msg_buffer[1024];

    printf("Enter msg: ");
    fgets(msg_buffer, sizeof(msg_buffer), stdin);
    if((send(sockfd, msg_buffer, strlen(msg_buffer), 0)) < 0) {
        perror("Error while sending message\n");
        exit(1);
    }

    memset(&msg_buffer, 0, sizeof(msg_buffer));
    if((recv(sockfd, msg_buffer, sizeof(msg_buffer), 0)) < 0) {
        perror("Error while receiving message\n");
        exit(1);
    }
```

```
    printf("Reversed string:\n%s\n", msg_buffer);
    close(sockfd);
    return 0;
}
```

Output :

```
Server connection successful
Enter msg: hola
Reversed string:
aloh
```

Output

Server Output:

```
Client connection successfull
Received string: hola
```

Client Output:

```
Server connection successfull
Enter msg: hola
Reversed string:
aloh
```

Result

The Program was executed successfully and output is verified.

EXP NO : 5 **Date : 31/12/2025**

UDP Client - Server Implementation

Aim

Write a UDP client-server program with the following functionality:
A. The client reads two numbers from the user and sends them to the server.
B. The server adds the numbers and sends back the sum to the client.

Algorithm

Server :

1. Start
2. Create a socket using `socket()` with `SOCK_DGRAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5001`, `INADDR_ANY`).
5. Bind the socket to the address using `bind()`.
6. Check for binding error.
7. Receive numbers from client into buffer using `recvfrom()`.
8. Check for receiving error.
9. Print the received data.
10. Tokenize the buffer using `strtok()` with space as delimiter.
11. Convert each token to integer using `atoi()` and accumulate into `sum`.
12. Repeat step 11 until no tokens remain.
13. Store the sum back into buffer using `sprintf()`.
14. Send the sum to the client using `sendto()`.
15. Check for sending error.
16. Print `"[+] Server data sent"`.

17. Close socket using `close(sockfd)`.
18. Stop

Client :

1. Start
2. Create a socket using `socket()` with `SOCK_DGRAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5001`, IP `127.0.0.1`).
5. Read numbers from the user using `fgets()`.
6. Send the numbers to the server using `sendto()`.
7. Check for sending error.
8. Print `"[+] Client data sent"`.
9. Receive the sum from the server into buffer using `recvfrom()`.
10. Check for receiving error.
11. Print `"[+] Server data received:"` followed by the sum.
12. Close socket using `close(sockfd)`.
13. Stop

Program Code

Server :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main(void) {
    struct sockaddr_in server_addr;
    int sockfd;
```

```
if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
    perror("Socket connection failed!\n");
    exit(1);
}

memset(&server_addr, '\0', sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(5001);
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);

socklen_t server_addr_len = sizeof(server_addr);

if (bind(sockfd, (struct sockaddr *)&server_addr,
   server_addr_len) < 0) {
    perror("Error binding connection!\n");
    exit(1);
}

char buffer[1024] = {0};

if (recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct
   sockaddr *)&server_addr, &server_addr_len) < 0) {
    perror("Error receiving content/data from client!\n");
    exit(1);
}

printf("[+] Client data received: %s\n", buffer);

char *token = strtok(buffer, " "); // Divide on space
int sum = 0;

while (token != NULL) {
    sum += atoi(token);
    token = strtok(NULL, " ");
}

sprintf(buffer, "%d", sum);

if (sendto(sockfd, buffer, strlen(buffer), 0, (struct
   sockaddr *)&server_addr, server_addr_len) < 0) {
```

```
        perror("Error sending content/data to client!\n");
        exit(1);
    }

    printf("[+] Server data sent\n");

    return 0;
}
```

Client :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main(void) {
    struct sockaddr_in server_addr;
    int sockfd;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket connection failed!\n");
        exit(1);
    }

    memset(&server_addr, '\0', sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5001);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    socklen_t server_addr_len = sizeof(server_addr);
    char msg_buffer[1024];

    /* Get numbers from user */
    printf("Enter numbers: ");
    fgets(msg_buffer, sizeof(msg_buffer), stdin);

    if (sendto(sockfd, msg_buffer, strlen(msg_buffer), 0, (struct
        sockaddr *)&server_addr, server_addr_len) < 0) {
```

```
        perror("Error sending content/data to server!\n");
        exit(1);
    }

    printf("[+] Client data sent\n");

    char buffer[1024] = {0};

    if (recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct
       sockaddr *)&server_addr, &server_addr_len) < 0) {
        perror("Error receiving content/data from server!\n");
        exit(1);
    }

    printf("[+] Server data received: %s\n", buffer);

    return 0;
}
```

Output

Server Output:

```
[+] Client data received: 10 10
[+] Server data sent
```

Client Output:

```
Enter numbers: 10 10
[+] Client data sent
[+] Server data received: 20
```

Result

The Program was executed successfully and output is verified.

EXP NO : 6 **Date : 23/02/2026**

File Transfer Protocol

Aim

Implementation of get and put commands of the File Transfer Protocol (FTP).

Algorithm

Server :

1. Start
2. Create a socket using `socket()` with `SOCK_STREAM`.
3. Check for socket creation error.
4. Initialize server address (`AF_INET`, port `5000`, `INADDR_ANY`).
5. Bind the socket to the address using `bind()`.
6. Check for binding error.
7. Listen for incoming connections using `listen()` with backlog 3.
8. Check for listening error.
9. Accept a client connection using `accept()`.
10. Check for accept error.
11. Receive command and filename from client into buffer using `recv()`.
12. Check for receiving error.
13. Print the received command string.
14. Parse buffer to extract `command` and `filename` using `sscanf()`.
15. If `command` is `"GET"`:
    (a) Open `filename` in read mode using `fopen()`.
    (b) Read file line by line using `fgets()` and send each line to client using `send()`.

(c) Close the file using `fclose()`.

(d) Print `"Successfully sent data to client"`.

16. Else if `command` is `"PUT"`:

(a) Open `filename` in write mode using `fopen()`.

(b) Receive file content from client in a loop using `recv()` and write to file using `fputs()`.

(c) Close the file using `fclose()`.

(d) Print `"File successfully uploaded to server!"`.

17. Close client socket using `close(clientfd)`.

18. Close server socket using `close(sockfd)`.

19. Stop

Client :

1. Start

2. Create a socket using `socket()` with `SOCK_STREAM`.

3. Check for socket creation error.

4. Initialize server address (`AF_INET`, port `5000`, IP `127.0.0.1`).

5. Connect to the server using `connect()`.

6. Check for connection error.

7. Prompt user to enter command and filename (e.g., `GET filename` or `PUT filename`).

8. Read input into buffer using `fgets()`.

9. Send buffer to server using `send()`.

10. Parse buffer to extract `command` and `filename` using `sscanf()`.

11. If `command` is `"GET"`:

(a) Open `filename` in write mode using `fopen()`.

(b) Receive file content from server in a loop using `recv()` and write to file using `fputs()`.

(c) Break loop if received bytes are less than buffer size.

(d) Close the file using `fclose()`.

(e) Print `"File downloaded successfully"`.

12. Else if `command` is `"PUT"`:

(a) Open `filename` in read mode using `fopen()`.

(b) Check if file exists; if not, print `"FILE NOT FOUND"` and exit.

(c) Read file line by line using `fgets()` and send each line to server using `send()`.

(d) Close the file using `fclose()`.

(e) Print `"File uploaded successfully"`.

13. Else print error for invalid command and exit.

14. Close socket using `close(sockfd)`.

15. Stop

Program Code

Server :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main() {
    struct sockaddr_in server_addr, client_addr;
    int sockfd, clientfd;

    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Error connecting socket\n");
        exit(1);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5000);
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
if((bind(sockfd, (struct sockaddr*) &server_addr, sizeof(
   server_addr))) < 0) {
    perror("Unable to bind the connection \n");
    exit(1);
}

if ((listen(sockfd, 3)) < 0) {
    perror("Unable to listen\n");
    exit(1);
}

socklen_t client_addr_len = sizeof(client_addr);
if((clientfd = accept(sockfd, (struct sockaddr*) &client_addr
   , &client_addr_len)) < 0) {
    perror("Unable to accept \n");
    exit(1);
}

char buffer[1024];
if((recv(clientfd, buffer, sizeof(buffer)-1, 0)) < 0) {
    perror("Error receiving content from client");
    exit(1);
}

printf("String from client: %s\n", buffer);
char command[5], filename[50], content[1024];
sscanf(buffer, "%s %s", command, filename);

if (strcmp(command, "GET") == 0) {
    FILE* fptr = fopen(filename, "r");
    while(fgets(content, sizeof(content), fptr)) {
        send(clientfd, content, strlen(content), 0);
    }
    fclose(fptr);
    printf("Successfully sent data to client\n");
}
else if (strcmp(command, "PUT") == 0) {
    FILE *fp = fopen(filename, "w");
    while((recv(clientfd, content, sizeof(content), 0)) > 0)
       {
        fputs(content, fp);
```

```
        }
        fclose(fp);
        printf("File successfully uploaded to server!\n");
    }

    return 0;
}
```

Client :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define size 1024

int main() {
    struct sockaddr_in server_addr;
    int sockfd;

    if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Unable to create socket\n");
        exit(0);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5000);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    socklen_t server_addr_len = sizeof(server_addr);
    if((connect(sockfd, (struct sockaddr *)&server_addr,
       server_addr_len)) < 0) {
        perror("Error connecting to server!\n");
        exit(1);
    }

    char buffer[1024], command[10], filename[10];
```

```
    // Read filename from user
    printf("Enter command (GET/PUT filename): ");
    fgets(buffer, sizeof(buffer), stdin);
    send(sockfd, buffer, sizeof(buffer), 0);
    sscanf(buffer, "%s %s", command, filename);

    if(strcmp(command, "GET") == 0) {
        FILE *fp = fopen(filename, "w");
        while((recv(sockfd, buffer, size, 0)) > 0) {
            fputs(buffer, fp);
            if(strlen(buffer) < size) break;
        }
        fclose(fp);
        printf("FILE downloaded successfully\n");
    } else if (strcmp(command, "PUT") == 0) {
        FILE *fp = fopen(filename, "r");
        if (fp == NULL) {
            printf("FILE NOT FOUND\n");
            exit(0);
        } else {
            while(fgets(buffer, size, fp)) {
                send(sockfd, buffer, strlen(buffer), 0);
            }
            fclose(fp);
            printf("File uploaded successfully\n");
        }
    } else {
        perror("Provide proper commands such as 'GET' or 'PUT'\n"
           );
        exit(1);
    }

    close(sockfd);
    return 0;
}
```

Output

Case 1: GET Command

Server :

```
String from client: GET hello.txt
Successfully sent data to client
```

Client :

```
Enter command (GET/ PUT filename): GET hello.txt
File downloaded successfully
```

Case 2: PUT Command

Server :

```
String from client: PUT upload.txt
File successfully uploaded to server!
```

Client :

```
Enter command (GET/ PUT filename): PUT upload.txt
File uploaded successfully
```

Result

The Program was executed successfully and output is verified.

EXP NO : 7 **Date : 23/02/2026**

Link State Routing

Aim

Implementation of Link state routing.

Algorithm

Link State Routing (Dijkstra's Algorithm) :

1. Start
2. Read the number of routers `n`.
3. Read the `n x n` cost matrix, where `9999` represents no direct link between routers.
4. Read the source router `src`.
5. Initialize distance array `dist[]` where `dist[i] = cost[src][i]` for all `i`.
6. Initialize visited array `s[]` with `false` for all routers.
7. Set `dist[src] = 0` and `s[src] = true`.
8. Repeat `n - 1` times:
   (a) Find the unvisited router `node` with the minimum value in `dist[]`.
   (b) Mark `s[node] = true` (add to visited set).
   (c) For each unvisited router `j`:
      i. If `dist[j] > dist[node] + cost[node][j]`, update `dist[j] = dist[node] + cost[node][j]`.
9. Print the shortest distance from source router `src` to all other routers.
10. Stop

Program

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

// Link state routing algorithm implementation
int main() {

  // Read the number of routers which are available
  int n ;
  printf("Enter the number of routers: ");
  scanf("%d", &n );

  // Read the cost matrix
  int cost[n][n];
  printf("Enter the cost matrix values => (9999 for no link): ");
  for(int i = 0; i < n; i++ ) {
    for(int j = 0; j< n; j++) {
      scanf("%d", &cost[i][j]);
    }
  }

  // Get the source router ie. From which router the distance is
     to be calculated.
  int src ;
  printf("Enter the source router: ");
  scanf("%d", &src);

  // ===> Dijkstra's algorithm

  // Initialize the dist and visited to default values
  int dist[n];
  bool s[n];
  for(int i = 0; i < n; i++) {
    dist[i]  = cost[src][i];
    s[i] = false;
  }

  dist[src] = 0;
  s[src] = true;
```

```
  for(int i = 1; i < n; i++ ){

  // Choose the node satisfying the conditions -> s[] should be
     false and dist[] should be minimum
    int min_value = INT_MAX, node ;
    for(int j = 0; j < n; j++ ){
      if(!s[j] && dist[j] < min_value){
        min_value = dist[j];
        node = j ;
      }
    }

    s[node] = true;

    // update the adjacent nodes dist with least cost

    for(int j = 0; j < n; j++ ){
      if(!s[j] && dist[j] > dist[node] + cost[node][j]) dist[j] =
          dist[node] + cost[node][j];
    }
  }

  // print the final shortest distance vector
  printf("\n\nDistance from the src router %d\n", src);
  for(int i = 0; i < n; i++) {
    printf("To router %d: %d\n" , i , dist[i]);
  }

  return 0 ;

}
```

Output

```
Enter the number of routers: 4
Enter the cost matrix values => (9999 for no link):
0 2 9999 1
2 0 1 3
9999 1 0 9999
1 3 9999 0
```

```
Enter the source router: 0

Distance from the src router 0
To router 0: 0
To router 1: 2
To router 2: 3
To router 3: 1
```

Result

The Program was executed successfully and output is verified.

EXP NO : 9 **Date : 11/03/2026**

Stop and Wait Algorithm

Aim

Implementation of Stop and Wait flow control algorithm.

Algorithm

Client :

1. Start
2. Read total number of frames `total_frames` from user.
3. Create a UDP socket using `socket()` with `SOCK_DGRAM`.
4. Initialize server address (`AF_INET`, port `8080`, `INADDR_ANY`).
5. Initialize `seq = 0`, `sent = 0`.
6. Repeat while `sent < total_frames`:
   (a) Print `"Sending frame: seq"`.
   (b) Send current `seq` to receiver using `sendto()`.
   (c) Wait for ACK/NAK response using `recvfrom()`.
   (d) If `ack == -1` (NAK received):
      i. Print `"Received NAK Resending frame seq"`.
      ii. Continue (do not increment `sent`).
   (e) Else:
      i. Print `"Received ACK: ack"`.
      ii. If `ack == (1 - seq)` (expected ACK):
         A. Set `seq = ack`.
         B. Increment `sent`.
      iii. Else: Print `"Unexpected ACK Resending"`.
7. Print `"All frames sent successfully!"`.
8. Close socket using `close(sockfd)`.
9. Stop

Server :

1. Start
2. Create a UDP socket using `socket()` with `SOCK_DGRAM`.
3. Initialize server address (`AF_INET`, port `8080`, `INADDR_ANY`).
4. Bind the socket using `bind()`.
5. Seed random number generator using `srand(time(0))`.
6. Initialize `expected = 0`.
7. Print `"Receiver ready..."`.
8. Repeat indefinitely:
   (a) Receive frame from sender using `recvfrom()`.
   (b) Print `"Received frame: frame"`.
   (c) If `rand() % 5 == 0` (simulate random error):
      i. Set `nak = -1`.
      ii. Print `"Error detected Sending NAK"`.
      iii. Send `nak` to sender using `sendto()`.
      iv. Continue.
   (d) Else if `frame == expected`:
      i. Print `"Correct frame"`.
      ii. Compute `ack = 1 - expected`.
      iii. Send `ack` to sender using `sendto()`.
      iv. Print `"ACK sent: ack"`.
      v. Update `expected = ack`.
   (e) Else:
      i. Print `"Wrong frame Sending NAK"`.
      ii. Send `nak = -1` to sender using `sendto()`.
9. Close socket using `close(sockfd)`.
10. Stop

Program Code

Client :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    socklen_t len = sizeof(server_addr);
    int seq = 0;   // Sequence number of current frame (0 or 1)
    int ack;       // To store ACK/NAK from receiver
    int total_frames;

    printf("Enter number of frames: ");
    scanf("%d", &total_frames);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0); // Create UDP socket

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY; // Sending to
       localhost

    int sent = 0;
    while (sent < total_frames) {
        printf("\nSending frame: %d\n", seq);

        // Send current frame
        sendto(sockfd, &seq, sizeof(seq), 0,
               (struct sockaddr*)&server_addr, len);

        // Wait for ACK/NAK
        recvfrom(sockfd, &ack, sizeof(ack), 0,
                 (struct sockaddr*)&server_addr, &len);

        if (ack == -1) {  // NAK received -> resend
```

```
            printf("Received NAK -> Resending frame %d\n", seq);
            continue;
        }

        printf("Received ACK: %d\n", ack);

        if (ack == (1 - seq)) {  // ACK matches expected -> move
           to next frame
            seq = ack;
            sent++;
        } else {  // Unexpected ACK -> resend current frame
            printf("Unexpected ACK Resending\n");
        }
    }

    printf("\nAll frames sent successfully!\n");
    close(sockfd);
    return 0;
}
```

Server :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <time.h>
#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t len = sizeof(client_addr);
    int frame;
    int expected = 0;   // expected frame number (0 or 1)

    sockfd = socket(AF_INET, SOCK_DGRAM, 0); // Create UDP socket

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
```

```
bind(sockfd, (struct sockaddr*)&server_addr, sizeof(
   server_addr)); // Bind socket

srand(time(0)); // Seed random number generator for error
   simulation

printf("Receiver ready...\n");

while (1) {
    // Receive frame from sender
    recvfrom(sockfd, &frame, sizeof(frame), 0,
             (struct sockaddr*)&client_addr, &len);

    printf("Received frame: %d\n", frame);

    // Simulate random error (send NAK)
    if (rand() % 5 == 0) {
        int nak = -1;
        printf("Error detected -> Sending NAK\n");
        sendto(sockfd, &nak, sizeof(nak), 0,
               (struct sockaddr*)&client_addr, len);
        continue;
    }

    if (frame == expected) {
        printf("Correct frame\n");
        int ack = 1 - expected;  // ACK for next expected
           frame
        sendto(sockfd, &ack, sizeof(ack), 0,
               (struct sockaddr*)&client_addr, len);
        printf("ACK sent: %d\n", ack);
        expected = ack;  // Update expected frame
    } else {
        printf("Wrong frame Sending NAK\n");
        int nak = -1;
        sendto(sockfd, &nak, sizeof(nak), 0,
               (struct sockaddr*)&client_addr, len);
    }
}
```

```
    close(sockfd);
    return 0;
}
```

Output

Sender (Client) :

```
Enter number of frames: 5

Sending frame: 0
Received ACK: 1

Sending frame: 1
Received NAK -> Resending frame 1

Sending frame: 1
Received ACK: 0

Sending frame: 0
Received ACK: 1

Sending frame: 1
Received ACK: 0

Sending frame: 0
Received ACK: 1

All frames sent successfully!
```

Receiver (Server) :

```
Receiver ready...
Received frame: 0
Correct frame
ACK sent: 1
Received frame: 1
Error detected -> Sending NAK
Received frame: 1
Correct frame
```

```
ACK sent: 0
Received frame: 0
Correct frame
ACK sent: 1
Received frame: 1
Correct frame
ACK sent: 0
Received frame: 0
Correct frame
ACK sent: 1
```

Result

The Program was executed successfully and output is verified.

EXP NO : 9 **Date : 04/03/2026**

Go Back N Algorithm

Aim

Implementation of Go Back - N flow control algorithm.

Algorithm

Server (Receiver):

1. Start
2. Accept `total_frames` as a command-line argument.
3. Create a UDP socket using `socket()` with `SOCK_DGRAM`.
4. Initialize server address (`AF_INET`, port `8080`, `INADDR_ANY`).
5. Bind the socket to the address using `bind()`.
6. Print `"Server started. Waiting for frames..."`.
7. Read the number of lost frames and their frame numbers from the user.
8. Initialize `next_frame = 0`, `corrupted_frame = -1`.
9. Repeat while `next_frame < total_frames`:
   (a) Compute `remaining_frames` as `min(WINDOW_SIZE, total_frames - next_frame)`.
   (b) For each frame in the current window:
      i. Receive a frame from the client using `recvfrom()`.
      ii. Print `"Received frame: recv_frame"`.
      iii. If `corrupted_frame == -1`, check if the frame is in the lost list; if so, set `corrupted_frame = recv_frame` and print `"Corrupted frame: corrupted_frame"`.
   (c) If `corrupted_frame != -1`:
      i. Set `next_frame = corrupted_frame` (go back).
      ii. Remove the frame from the lost list.
      iii. Reset `corrupted_frame = -1`.

(d) Else: advance `next_frame += remaining_frames`.

(e) Send cumulative ACK `next_frame` to the client using `sendto()`.

(f) Print `"Sent ACK: next_frame"`.

10. Close the socket using `close()`.
11. Print `"Server finished."`.
12. Stop

Client (Sender):

1. Start
2. Accept `total_frames` as a command-line argument.
3. Create a UDP socket using `socket()` with `SOCK_DGRAM`.
4. Initialize server address (`AF_INET`, port `8080`, `INADDR_ANY`).
5. Initialize `next_frame = 0`.
6. Repeat while `next_frame < total_frames`:

(a) Compute `remaining_frames` as `min(WINDOW_SIZE, total_frames - next_frame)`.

(b) For each frame in the window:

i. Print `"Sent frame: next_frame"`.

ii. Send `next_frame` to the server using `sendto()`.

iii. Increment `next_frame`.

(c) Wait for cumulative ACK from the server using `recvfrom()`.

(d) Print `"Received ACK: ack_frame"`.

(e) Set `next_frame = ack_frame` (slide window based on ACK).

7. Close the socket using `close()`.
8. Print `"Client finished."`.
9. Stop

Program Code

Server (Receiver):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define WINDOW_SIZE 4
#define MAX_FRAMES 100

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <total_frames>\n", argv[0]);
        exit(1);
    }
    int total_frames = atoi(argv[1]);
    int lost_frames[MAX_FRAMES], num_lost = 0;
    int next_frame = 0;
    int corrupted_frame = -1;
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(1);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(8080);

    if (bind(sockfd, (struct sockaddr *)&server_addr,
             sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(1);
    }
```

```
printf("Server started. Waiting for frames...\n");

printf("Enter the number of lost frames: ");
scanf("%d", &num_lost);
if (num_lost > 0) {
    printf("Enter the frame numbers of lost frames: ");
    for (int i = 0; i < num_lost; i++)
        scanf("%d", &lost_frames[i]);
}

while (next_frame < total_frames) {
    int remaining_frames = ((next_frame + WINDOW_SIZE) <
       total_frames)
                          ? WINDOW_SIZE : total_frames -
                             next_frame;

    for (int i = 0; i < remaining_frames; i++) {
        int recv_frame;
        recvfrom(sockfd, &recv_frame, sizeof(recv_frame), 0,
                 (struct sockaddr *)&client_addr, &addr_len);
        printf("Received frame %d\n", recv_frame);

        if (corrupted_frame == -1) {
            for (int j = 0; j < num_lost; j++) {
                if (recv_frame == lost_frames[j]) {
                    corrupted_frame = recv_frame;
                    printf("Corrupted frame %d\n",
                       corrupted_frame);
                    break;
                }
            }
        }
    }

    if (corrupted_frame != -1) {
        next_frame = corrupted_frame;
        for (int i = 0; i < num_lost; i++) {
            if (lost_frames[i] == corrupted_frame) {
                for (int j = i; j < num_lost - 1; j++)
                    lost_frames[j] = lost_frames[j + 1];
```

```
                    num_lost--;
                    break;
                }
            }
            corrupted_frame = -1;
        } else {
            next_frame += remaining_frames;
        }

        sendto(sockfd, &next_frame, sizeof(next_frame), 0,
               (struct sockaddr *)&client_addr, addr_len);
        printf("Sent ACK %d\n", next_frame);
    }

    close(sockfd);
    printf("Server finished.\n");
    return 0;
}
```

Client (Sender):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define WINDOW_SIZE 4

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <total_frames>\n", argv[0]);
        exit(1);
    }
    int total_frames = atoi(argv[1]);
    int next_frame = 0;
    int ack_frame;
    int sockfd;
    struct sockaddr_in server_addr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
```

```
        exit(1);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);
    server_addr.sin_addr.s_addr = INADDR_ANY;

    while (next_frame < total_frames) {
        int remaining_frames = ((next_frame + WINDOW_SIZE) <
           total_frames)
                               ? WINDOW_SIZE : total_frames -
                                  next_frame;

        for (int i = 0; i < remaining_frames; i++) {
            printf("Sent frame %d\n", next_frame);
            sendto(sockfd, &next_frame, sizeof(next_frame), 0,
                   (struct sockaddr *)&server_addr, sizeof(
                      server_addr));
            next_frame++;
        }
        recvfrom(sockfd, &ack_frame, sizeof(ack_frame), 0, NULL,
           NULL);
        printf("Received ACK %d\n", ack_frame);

        next_frame = ack_frame;
    }
    close(sockfd);
    printf("Client finished.\n");
    return 0;
}
```

Output

Server (Receiver):

```
Server started. Waiting for frames...
Enter the number of lost frames: 1
Enter the frame numbers of lost frames: 4
Received frame 0
Received frame 1
```

```
Received frame 2
Received frame 3
Sent ACK 4
Received frame 4
Corrupted frame 4
Received frame 5
Received frame 6
Received frame 7
Sent ACK 4
Received frame 4
Received frame 5
Received frame 6
Received frame 7
Sent ACK 8
Server finished.
```

Client (Sender):

```
Sent frame 0
Sent frame 1
Sent frame 2
Sent frame 3
Received ACK 4
Sent frame 4
Sent frame 5
Sent frame 6
Sent frame 7
Received ACK 4
Sent frame 4
Sent frame 5
Sent frame 6
Sent frame 7
Received ACK 8
Client finished.
```

Result

The Program was executed successfully and output is verified.

EXP NO : 10 **Date : 04/03/2026**

Familiarization of Wireshark

Aim

To familiarize with basic networking concepts and commonly used networking commands.

What is Wireshark ?

- Wireshark is a network packet analyzer.
- A network packet analyzer presents captured packet data in as much detail as possible.

Wireshark Display Filters

- Display filters are used to selectively display packets from the captured packets.

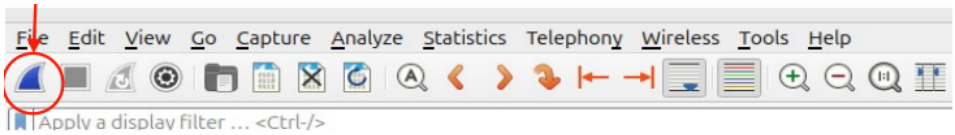
Example	Description
ip.dest == 192.168.1.1	Destination ip address equal to 192.168.1.1
ip.src != 192.168.1.1	Source ip address not equal to 192.168.1.1
ip.addr == 10.10.50.1	IP address equal to 10.10.50.1
ip.addr >= 10.10.50.1 and ip.addr <= 10.10.50.100	Filter by IP range
tcp.port == 25	Filter by port
tcp.dstport == 23	Filter by destination port
tcp.flags.syn == 1	Filter SYN flag
ip.host = hostname	Host name filter

Wireshark Capture Filters

- Capture filters are used to reduce the amount of packet data captured.

host 172.18.5.4	Capture only traffic to or from IP address 172.18.5.4
net 192.168.0.0 mask 255.255.255.0 or net 192.168.0.0/24	Capture traffic to or from a range of IP addresses
port 53	Capture only DNS packets
host www.example.com and not (port 80 or port 25)	Capture non-HTTP and non-SMTP traffic on your server (both are equivalent)
port not 53 and not arp	Capture except all ARP and DNS traffic.

1. Run sudo wireshark.
2. Choose the interface to capture the packets.
3. Click to start capturing.



4. Click the 'stop' button.
5. Observe the fields in the packet list pane :
   (a) Source address, destination address, Protocol, packet length
6. Fliter pane
   (a) Apply filter 'http'.
      i. If no packets are found, type neverssl.com in the browser while continuing capture.
   (b) Apply filter tcp.port == 443.
   (c) Apply filter udp.port == 443.
7. Filter all DNS packets.
   (a) Do they use TCP or UDP?
   Answer: UDP
   (b) Which port number is used by DNS?
   Answer: 53
   (c) Identify DNS request and DNS reply packets.
   Answer: In the Info column, DNS query packets are labeled as `Standard query` and DNS reply packets are labeled as `Standard query response`. The Flags field in the DNS header distinguishes them the QR bit is `0` for a request and `1` for a reply.
   (d) What is the IP address of the DNS server?
   Answer: The DNS server IP address is found in the *Destination* field of a DNS query packet (or the *Source* field of a DNS reply). It is typically the default gateway or ISP-assigned DNS server (e.g., `8.8.8.8` for Google DNS or `192.168.x.x` for a local resolver).

(e) What is the type of the resource record used?
Answer: A (IPv4)

8. Identify the packet headers and analyze their contents at the following layers.

(a) Ethernet
Answer: The Ethernet header contains the source MAC address, destination MAC address, and EtherType field (e.g., `0x0800` for IPv4). It operates at Layer 2 (Data Link layer).

(b) IP
Answer: The IP header contains the source IP address, destination IP address, TTL (Time To Live), protocol number (e.g., `6` for TCP, `17` for UDP), and header checksum. It operates at Layer 3 (Network layer).

(c) TCP
Answer: The TCP header contains the source port, destination port, sequence number, acknowledgment number, flags (SYN, ACK, FIN, etc.), window size, and checksum. It operates at Layer 4 (Transport layer).

Result

The network packets were captured and analyzed using Wireshark and different protocol fields and filters were studied successfully.

EXP NO : 11 **Date : 30/03/2026**

Multi-client UDP Server

Aim

Implementation of Multi-client UDP server using epoll.

Algorithm

Server :

1. Start.
2. Create UDP Socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize Server address
   (a) `sin_family = AF_INET`
   (b) `sin_addr = INADDR_ANY`
   (c) `sin_port = htons(PORT)`
4. Bind the socket using `bind()`.
5. Set Non-Blocking Mode using `fcntl()` to enable `O_NONBLOCK`.
6. Call `epoll_create1()` to create epoll instance.
7. Register Socket with Epoll
   (a) Add socket using `epoll_ctl()`.
   (b) Monitor `EPOLLIN` events.
8. Repeat:
   (a) Call `epoll_wait()` to wait for events.
   (b) For each event:
      i. If socket is ready:
         - Call `recvfrom()`.
         - If no data (`EAGAIN`), break.
         - Print client IP, port, message.

- Send response using `sendto()`.

9. Close socket and epoll descriptor.
10. Stop.

Client :

1. Start.
2. Create a UDP socket using `socket()`.
3. Initialize Server Address:
   (a) `sin_family = AF_INET`
   (b) `sin_port = htons(PORT)`
   (c) Convert IP using `inet_pton()`.
4. Repeat until user exits:
   (a) Read input using `fgets()`.
   (b) If input is "exit" → terminate.
   (c) Send message using `sendto()`.
   (d) Wait for response using `recvfrom()`.
   (e) Print server response.
5. Close the socket.
6. Stop.

Program Code

Server :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <sys/epoll.h>
#include <errno.h>
```

```
#define PORT 9090
#define MAX 10
#define SIZE 1024

int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags == -1) {
        perror("fcntl GETFL failed");
        return -1;
    }
    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) == -1) {
        perror("fcntl SETFL failed");
        return -1;
    }
    return 0;
}

int main() {
    int sockfd, epfd;
    struct sockaddr_in server_addr = {0};
    struct sockaddr_in client_addr = {0};
    socklen_t client_len;
    char buffer[SIZE];
    struct epoll_event ev, events[MAX];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd == -1) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(sockfd, (struct sockaddr*)&server_addr, sizeof(
       server_addr)) == -1) {
        perror("bind failed");
    }
```

```
if (set_nonblocking(sockfd) == -1) {
    close(sockfd);
    exit(EXIT_FAILURE);
}

epfd = epoll_create1(0);
if (epfd == -1) {
    perror("epoll_create1 failed");
    close(sockfd);
    exit(EXIT_FAILURE);
}

ev.events = EPOLLIN;
ev.data.fd = sockfd;
if (epoll_ctl(epfd, EPOLL_CTL_ADD, sockfd, &ev) == -1) {
    perror("epoll_ctl failed");
    close(sockfd);
    close(epfd);
    exit(EXIT_FAILURE);
}

printf("UDP Server running on port %d\n", PORT);

while (1) {
    int nfds = epoll_wait(epfd, events, MAX, -1);
    if (nfds == -1) {
        perror("epoll_wait failed");
        break;
    }
    for (int i = 0; i < nfds; i++) {
        if (events[i].data.fd == sockfd) {
            while (1) {
                client_len = sizeof(client_addr);
                ssize_t len = recvfrom(sockfd, buffer, SIZE -
                    1, 0, (struct sockaddr*)&client_addr, &
                   client_len);
                if (len == -1) {
                    if (errno == EAGAIN || errno ==
                       EWOULDBLOCK)
                        break;
                    perror("recvfrom failed");
```

```
                        break;
                    }
                    buffer[len] = '\0';
                    printf("Client %s:%d     %s\n", inet_ntoa(
                       client_addr.sin_addr), ntohs(client_addr.
                       sin_port), buffer);

                    if (sendto(sockfd, buffer, len, 0, (struct
                       sockaddr*)&client_addr, client_len) == -1)
                        {
                        perror("sendto failed");
                    }
                }
            }
        }
    }

    close(sockfd);
    close(epfd);
    return 0;
}
```

Client :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <errno.h>

#define PORT 9090
#define SIZE 1024

int main() {
    int sockfd;
    struct sockaddr_in server_addr = {0};
    char buffer[SIZE];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd == -1) {
```

```
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr) <=
        0) {
        perror("inet_pton failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("Enter messages:\n");
    while (1) {
        if (fgets(buffer, SIZE, stdin) == NULL) {
            perror("fgets failed");
            break;
        }
        if (strncmp(buffer, "exit", 4) == 0)
            break;

        ssize_t sent = sendto(sockfd, buffer, strlen(buffer), 0,
           (struct sockaddr*)&server_addr, sizeof(server_addr));
        if (sent == -1) {
            perror("sendto failed");
            continue;
        }

        ssize_t len = recvfrom(sockfd, buffer, SIZE - 1, 0, NULL,
            NULL);
        if (len == -1) {
            perror("recvfrom failed");
            continue;
        }

        buffer[len] = '\0';
        printf("Server reply: %s", buffer);
    }
```

```
    close(sockfd);
    return 0;
}
```

Output

Server Output:

```
UDP Server running on port 9090
Client 127.0.0.1:53214  Hello Server!
Client 127.0.0.1:53214  Testing UDP epoll.
```

Client Output:

```
Enter messages:
Hello Server!
Server reply: Hello Server!
Testing UDP epoll.
Server reply: Testing UDP epoll.
exit
```

Result

The Program was executed successfully and output is verified.